

MICROSOFT FABRIC

# DP-700 Study Guide

Implementing Data Engineering Solutions Using Microsoft Fabric

*A concise revision guide for data professionals*

---

**Prepared by Jitendra Singh Malik**

August 2026

## **Independent study resource**

These notes were created while preparing for the DP-700 exam. They are not official Microsoft exam material. Features, previews, licensing and user-interface details may change, so validate current behaviour using Microsoft Learn and product documentation.

## How to use this guide

This guide condenses the original preparation notes into a structured reference for LinkedIn data professionals. It focuses on selecting the right Fabric component, understanding core architecture, and revising common exam distinctions.

- Read chapters 1–4 to build the platform foundation.
- Use the comparison tables to revise scenario-based decisions.
- Review the “Common confusion” callouts before practice exams.
- Use the final quick-reference section as a last-day revision sheet.

## Contents

1. Fabric foundations and OneLake
2. Data ingestion and orchestration
3. Apache Spark and notebooks
4. Lakehouse, Delta Lake and medallion architecture
5. Fabric Data Warehouse
6. Real-Time Intelligence
7. Security and access control
8. Monitoring and performance
9. CI/CD and lifecycle management
10. Final revision tables and scenario guide

### Source note

The guide is based on the supplied DP-700 preparation document and preserves its terminology, examples and learning emphasis. Repeated material has been consolidated and language has been polished for publication.

## CHAPTER 1

# Fabric foundations and OneLake

*Understand the unified platform before choosing individual workloads.*

## Microsoft Fabric at a glance

Microsoft Fabric is an all-in-one analytics platform that brings data ingestion, engineering, warehousing, real-time analytics, data science and Power BI into one SaaS environment. Its shared storage foundation is OneLake.

| Workload               | Primary purpose                                               |
|------------------------|---------------------------------------------------------------|
| Data Factory           | Ingest, transform and orchestrate data.                       |
| Data Engineering       | Build lakehouses and Spark-based transformation workflows.    |
| Data Warehouse         | Create relational analytical stores with transactional T-SQL. |
| Real-Time Intelligence | Ingest, store, query and act on streaming events.             |
| Data Science           | Explore data and build machine-learning solutions.            |
| Power BI               | Model, visualise and deliver business insights.               |

## OneLake: one logical data lake

OneLake is the unified storage layer for Fabric. Data is stored in open formats such as Delta-Parquet so multiple engines—Spark, SQL and Power BI—can work with the same data without unnecessary duplication.

- Shortcuts reference data in locations such as Amazon S3 or Azure Data Lake without copying it.
- Workspaces organise items for collaboration, access control and lifecycle management.
- The OneLake catalog supports discovery, metadata and governance across the data estate.

### Memory tip

Think of OneLake as the shared foundation, while Fabric workloads are different engines and experiences operating on that foundation.

## How data teams work together

| Role               | Typical Fabric responsibility                                           |
|--------------------|-------------------------------------------------------------------------|
| Data engineer      | Ingests and transforms data using pipelines, notebooks and lakehouses.  |
| Data scientist     | Uses notebooks, Spark and Python for exploration and machine learning.  |
| Data analyst       | Builds semantic models and reports, often using Direct Lake.            |
| Analytics engineer | Curates reliable business-ready data and reusable semantic definitions. |

## Workspaces and platform enablement

Fabric must be enabled by an administrator. Workspaces then act as collaboration containers and can provide lineage, Git integration and managed compute. Access can be granted broadly through workspace roles or narrowly through item permissions.

### Common confusion

A workspace is an organisational and security boundary. OneLake is the storage foundation. A lakehouse, warehouse, eventhouse or semantic model is an item inside a workspace.

## CHAPTER 2

# Data ingestion and orchestration

*Choose between Copy activity, Dataflow Gen2 and pipelines.*

## Dataflow Gen2

Dataflow Gen2 is a low-code, cloud-based ETL tool built on Power Query Online. Its common workflow is Connect → Transform → Land.

| Strengths                                               | Limitations                                                        |
|---------------------------------------------------------|--------------------------------------------------------------------|
| Reusable visual transformation logic                    | Not intended as long-term storage.                                 |
| Accessible to low-code users                            | Does not itself provide row-level security.                        |
| Can reduce repeated load on source systems              | Requires a Fabric capacity workspace.                              |
| Can land data in a Lakehouse, Warehouse or SQL Database | Complex engineering logic may be better suited to Spark notebooks. |

### Key interface areas

| Area           | Purpose                                               |
|----------------|-------------------------------------------------------|
| Ribbon         | Connectors, transformations and destination settings. |
| Queries pane   | Lists sources and supports Duplicate or Reference.    |
| Diagram view   | Shows the transformation flow visually.               |
| Data preview   | Displays sample results after each step.              |
| Query settings | Shows applied steps and transformation history.       |

- Use Reference when one source query feeds multiple downstream tables.
- Use query folding when supported so transformations are pushed to the source system.

## Data pipelines

A data pipeline is a sequence of activities used to orchestrate an end-to-end process. It can run manually or on a schedule and is similar in concept to Azure Data Factory.

| Activity type                   | Examples                                                        |
|---------------------------------|-----------------------------------------------------------------|
| Data movement or transformation | Copy Data, Dataflow Gen2, Notebook, Stored Procedure or Script. |
| Control flow                    | If condition, ForEach, Wait, variables and parameters.          |
| Metadata and validation         | Check whether files or objects exist before processing.         |

| Activity type       | Examples                                             |
|---------------------|------------------------------------------------------|
| Operational control | Retries, dependencies, schedules and run monitoring. |

## Parameters and reusability

Parameters allow one pipeline to process different files, folders, dates, tables or business entities. For example, the same pipeline can load January, February and March by changing a Month parameter at runtime.

### Exam distinction

A parameter is supplied to make the pipeline reusable. A variable stores or updates a value during a specific pipeline run.

## Copy Data activity

Copy Data is designed for efficient movement of data from a source to a destination, especially when little or no transformation is required. It can be used alone or as one step in a larger pipeline.

| Use Copy Data when...                           | Use Dataflow Gen2 when...                                    |
|-------------------------------------------------|--------------------------------------------------------------|
| Data should move largely as-is.                 | Data must be cleaned, reshaped or filtered during ingestion. |
| Transformation will happen later in a notebook. | Multiple sources must be merged visually.                    |
| High-performance data movement is the priority. | A Power Query-style low-code experience is preferred.        |

## Typical orchestrated sequence

11. Delete or clear the previous staging data.
12. Copy the new raw batch into OneLake.
13. Run a notebook or SQL script to transform the data.
14. Refresh or update downstream analytical objects.

### Worker vs manager

Dataflow Gen2 performs transformation work. A pipeline coordinates, schedules and connects Dataflows, notebooks, scripts and control-flow activities.

## CHAPTER 3

# Apache Spark and notebooks

*Use distributed processing for large-scale transformations.*

## Spark architecture in Fabric

Apache Spark is an open-source parallel-processing framework for large-scale analytics. Fabric provides an integrated environment for Spark-based ingestion, transformation and analysis within a lakehouse.

| Component                | Role                                                                |
|--------------------------|---------------------------------------------------------------------|
| Head node / driver       | Coordinates the application and distributed processing.             |
| Worker nodes / executors | Perform the actual processing tasks.                                |
| Spark pool               | The compute cluster used to execute workloads.                      |
| Runtime                  | Defines Spark, Delta Lake and Python versions.                      |
| Environment              | Stores libraries and custom configuration for consistent execution. |

## Working with DataFrames

A DataFrame is Spark's primary structured-data abstraction. It supports chained transformations and can load from files or tables.

| Spark operation         | SQL equivalent                                 |
|-------------------------|------------------------------------------------|
| select()                | SELECT column list                             |
| where()                 | WHERE condition                                |
| groupBy().count()       | GROUP BY with aggregation                      |
| write.mode("overwrite") | Replace the existing target data               |
| partitionBy()           | Physically organise files by a selected column |

### PySpark

```
df = spark.read.format("csv").option("header", True).load(source_path)
result = df.where("SalesAmount > 100").select("CustomerID", "SalesAmount")
```

### Schema choice

Schema inference is convenient, but an explicit schema is more reliable and can be more efficient for repeatable production workloads.

## Spark SQL and the catalog

Spark SQL allows relational querying inside notebooks. A temporary view is session-based, while a table saved to the catalog is persistent.

| SQL Server concept    | Spark SQL equivalent                                               |
|-----------------------|--------------------------------------------------------------------|
| Local temporary table | Temporary view created with <code>createOrReplaceTempView</code> . |
| Permanent table       | Managed table, commonly stored in Delta format.                    |
| External table        | Metadata in the catalog with files stored elsewhere.               |
| SSMS query tab        | A notebook cell using the <code>%%sql</code> magic command.        |

### PySpark + Spark SQL

```
df.createOrReplaceTempView("sales_view")
summary = spark.sql("SELECT CustomerID, SUM(SalesAmount) AS TotalSales FROM sales_view GROUP BY CustomerID")
```

## Saving and partitioning data

- Parquet is a compressed columnar format suited to analytical workloads.
- Delta tables add transactions, schema controls and version history on top of Parquet files.
- Partitioning can improve selective queries but should be reserved for large tables and sensible low-cardinality filter columns.

## Notebook visualisation

Fabric notebooks can display DataFrames as built-in charts. For more customised visuals, Spark results are often converted to Pandas and plotted using Python libraries. Use this for exploration; use Power BI for governed business reporting.



## CHAPTER 4

# Lakehouse, Delta Lake and medallion architecture

*Combine flexible data-lake storage with reliable analytical tables.*

## Lakehouse fundamentals

A lakehouse combines the flexible storage of a data lake with structured querying and relational features associated with a data warehouse. In Fabric, it supports both Spark and a SQL analytics endpoint.

| Lakehouse area         | Purpose                                                              |
|------------------------|----------------------------------------------------------------------|
| Files                  | Raw or semi-structured files such as CSV, JSON, images and Parquet.  |
| Tables                 | Delta tables with metadata, ACID transactions and schema support.    |
| SQL analytics endpoint | Read-only T-SQL access to lakehouse Delta tables.                    |
| Spark notebooks        | Engineering, advanced transformation and machine-learning workloads. |
| Semantic model         | Relationships, measures and business-ready reporting definitions.    |

### Common confusion

The SQL analytics endpoint is read-only for lakehouse tables. A Fabric Warehouse supports read-and-write transactional T-SQL.

## Delta Lake structure

A Delta table is a schema abstraction over Parquet data files plus a transaction log. The `_delta_log` folder records changes and enables reliable table operations.

| Capability          | Why it matters                                              |
|---------------------|-------------------------------------------------------------|
| ACID transactions   | Protects consistency during concurrent operations.          |
| Schema enforcement  | Prevents incompatible structures from being written.        |
| CRUD operations     | Supports insert, update and delete semantics.               |
| Time travel         | Allows access to previous table versions.                   |
| Batch and streaming | The same table can participate in both processing patterns. |
| Open storage        | Parquet-based data remains accessible to multiple engines.  |

## Managed and external Delta tables

| Managed table                                      | External table                                                       |
|----------------------------------------------------|----------------------------------------------------------------------|
| Metadata and data are managed by the lakehouse.    | Metadata is managed in the catalog, but files are stored separately. |
| Dropping the table can remove the underlying data. | Dropping the definition does not remove the external files.          |
| Convenient for standard lakehouse tables.          | Useful when storage location must remain independent.                |

## Optimising Delta tables

| Feature        | Primary purpose                              | When to use                                                   |
|----------------|----------------------------------------------|---------------------------------------------------------------|
| Optimize Write | Reduces small files during writes.           | Generally enabled by default in Fabric.                       |
| OPTIMIZE       | Compacts existing small files.               | After large loads or when file fragmentation affects reads.   |
| V-Order        | Improves read efficiency for Fabric engines. | Frequently queried reporting tables.                          |
| VACUUM         | Removes obsolete unreferenced files.         | Storage maintenance, with awareness of time-travel retention. |
| Partitioning   | Enables data skipping by folder layout.      | Very large tables with logical filter columns.                |

### Important

VACUUM removes old files outside the retention period and can prevent time travel to older versions.

## Time travel and streaming

### SQL and PySpark

```
DESCRIBE HISTORY sales;

# Read a previous version
historical_df = (spark.read.format("delta")
    .option("versionAsOf", 0)
    .table("sales"))
```

Spark Structured Streaming treats incoming data as a continuously growing DataFrame. Delta can act as both source and sink. A checkpoint location stores stream progress so processing can recover after failure.

## Medallion architecture

| Layer  | Purpose                                                    |
|--------|------------------------------------------------------------|
| Bronze | Raw source-aligned landing data with minimal modification. |

| Layer  | Purpose                                                                  |
|--------|--------------------------------------------------------------------------|
| Silver | Validated, cleaned, deduplicated and integrated data.                    |
| Gold   | Business-ready, enriched or aggregated data for reporting and analytics. |

- Use Dataflows Gen2 for simpler low-code transformations.
- Use notebooks for large-scale or complex transformations.
- Use pipelines to orchestrate movement between layers.

#### Design principle

The medallion pattern organises progressive data quality. It does not replace dimensional modelling; the Gold layer can still contain fact and dimension structures.

## CHAPTER 5

# Fabric Data Warehouse

*Design, load, query and secure relational analytical data.*

## Warehouse fundamentals

A Fabric Data Warehouse is a fully managed relational analytical store built on OneLake. It supports transactional T-SQL, ACID behaviour and open Delta storage.

| Warehouse                                                             | SQL analytics endpoint                                  |
|-----------------------------------------------------------------------|---------------------------------------------------------|
| Full read-and-write T-SQL including INSERT, UPDATE, DELETE and MERGE. | Read-only T-SQL experience over lakehouse Delta tables. |
| Native warehouse tables.                                              | Lakehouse tables surfaced through SQL.                  |
| Suitable for relational dimensional models and SQL-led ETL.           | Suitable for querying data engineered in the lakehouse. |

## Dimensional modelling

| Object                   | Purpose                                                                |
|--------------------------|------------------------------------------------------------------------|
| Fact table               | Stores quantitative measurements such as sales amount or quantity.     |
| Dimension table          | Provides descriptive context such as customer, product, store or date. |
| Surrogate key            | Warehouse-generated identifier with no business meaning.               |
| Business / alternate key | Source-system identifier retained for traceability and lookups.        |
| Star schema              | Fact table connected directly to denormalised dimensions.              |
| Snowflake schema         | Dimensions further normalised into related tables.                     |

## Slowly changing dimensions

| Type   | Behaviour                                                   |
|--------|-------------------------------------------------------------|
| Type 0 | Attributes do not change.                                   |
| Type 1 | Overwrite the existing value; history is not retained.      |
| Type 2 | Insert a new row for each change; full history is retained. |
| Type 3 | Store limited history in additional columns.                |

Fact tables are generally loaded after dimensions because incoming business keys must be mapped to the correct dimension surrogate keys.

## Loading strategies

| Strategy         | Description                                                                               |
|------------------|-------------------------------------------------------------------------------------------|
| Full load        | Truncate and reload the complete target. Simpler, but more resource intensive.            |
| Incremental load | Process only new or changed records. More efficient, but requires change-detection logic. |
| Staging          | Land source-shaped data in auxiliary tables before applying business rules.               |
| COPY statement   | Bulk load supported external files into warehouse tables.                                 |
| CTAS             | Create a new table from a SELECT result.                                                  |
| INSERT...SELECT  | Append query results into an existing target table.                                       |

### Common confusion

Data ingestion moves raw data into the analytical platform. Data loading places prepared data into final warehouse structures such as dimensions and facts.

## Querying and transformation

- The SQL query editor provides T-SQL, IntelliSense and saved workspace queries.
- The visual query editor provides a no-code canvas and generates SQL in the background.
- Views standardise joins, filters and column selections.
- Stored procedures encapsulate repeatable operational data tasks.

### Useful analytical functions

| Function              | Behaviour                                                   |
|-----------------------|-------------------------------------------------------------|
| ROW_NUMBER()          | Unique sequential row number within a partition.            |
| RANK()                | Equal values share rank; subsequent ranks contain gaps.     |
| DENSE_RANK()          | Equal values share rank; no gaps follow ties.               |
| NTILE(n)              | Divides rows into ranked groups.                            |
| APPROX_COUNT_DISTINCT | Returns a fast estimate for large distinct-count workloads. |

## Semantic modelling and Direct Lake

Clean business-friendly names, descriptions, relationships and reusable measures improve usability for both people and AI-assisted experiences. Direct Lake allows Power BI semantic models to read OneLake data without a traditional import copy.

## External client tools

- Copy the warehouse SQL connection string from Fabric.

- Use Microsoft Entra ID authentication; traditional SQL authentication is not supported.
- Specify the warehouse database name when required by the client tool.
- Standard ODBC and OLE DB tools can connect when network access permits TCP port 1433.

## CHAPTER 6

# Real-Time Intelligence

*Process, analyse and act on data in motion.*

## Real-time analytics concepts

Real-time analytics processes and acts on data as it is generated, usually within seconds or minutes. An event is one digital occurrence; a stream is the continuous chronological sequence of events.

| Capability                           | Fabric component              |
|--------------------------------------|-------------------------------|
| Discover streaming sources           | Real-Time Hub                 |
| Capture, transform and route events  | Eventstream                   |
| Store high-velocity time-series data | Eventhouse with KQL databases |
| Write and save analytical queries    | KQL queryset                  |
| Display live operational metrics     | Real-Time Dashboard           |
| Trigger actions from conditions      | Activator                     |

## Eventstream

Eventstream is a no-code visual experience for capturing, transforming and routing data in motion.

| Stage           | Examples                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------|
| Sources         | Azure Event Hubs, IoT Hub, Service Bus, Kafka, MQTT, Google Cloud Pub/Sub and Fabric events. |
| Transformations | Filter, manage fields, aggregate, group by time window, union, join and expand.              |
| Destinations    | Eventhouse, Lakehouse, Activator, derived stream or custom endpoint.                         |

### Scenario guide

Use Eventstream when data must be transformed or routed while in transit. Use direct ingestion when the priority is rapidly landing high-volume data into a KQL database and transforming it afterwards with update policies.

## Eventhouse and KQL databases

An Eventhouse is a container for real-time data stores. Its KQL databases are optimised for append-heavy, time-series workloads and automatically organise data for efficient recent-data analysis.

| KQL practice | Reason |
|--------------|--------|
|--------------|--------|

| KQL practice                                | Reason                                                                 |
|---------------------------------------------|------------------------------------------------------------------------|
| Filter early, especially by time            | Reduces scanned data and uses indexing effectively.                    |
| Project only required columns               | Reduces memory and processing overhead.                                |
| Use the smaller input on the left of a join | Improves join efficiency.                                              |
| Use limit for exploration                   | Prevents unnecessary large result displays.                            |
| Use materialized views                      | Precomputes recurring aggregations while incorporating new delta data. |
| Use stored functions                        | Standardises reusable and parameterised KQL logic.                     |

## KQL

```
stock
| where ["time"] > ago(5m)
| summarize avgPrice = avg(todouble(bidPrice)) by symbol
```

## Real-Time Dashboards

- Tiles execute KQL queries and refresh automatically.
- Base queries allow several tiles to reuse the same core query logic.
- Parameters make dashboards interactive for viewers.
- Multiple pages organise complex operational monitoring scenarios.

## Activator

Activator detects conditions in changing data and performs actions. It follows a Connect → Monitor → Act model.

| Concept  | Meaning                                                       |
|----------|---------------------------------------------------------------|
| Object   | A real-world entity such as a device, package or customer.    |
| Property | A tracked attribute such as temperature or location.          |
| Event    | An incoming record that updates the object state.             |
| Rule     | The condition, threshold or pattern that triggers action.     |
| Action   | Email, Teams, Power Automate, notebook or pipeline execution. |

### Rule design

Use summarisation windows and duration conditions to avoid triggering on short-lived noise. Property filters can narrow a rule to selected objects, priorities or locations.



## CHAPTER 7

# Security and access control

*Apply least privilege across workspace, item, compute and data layers.*

## Security evaluation model

15. Microsoft Entra ID authenticates the user.
16. Fabric access determines whether the user can access the environment and item.
17. Data security determines which tables, rows, columns or files are available.

## Workspace roles

| Role        | Capability                                                                                      |
|-------------|-------------------------------------------------------------------------------------------------|
| Admin       | Full control over content, data, settings and permissions.                                      |
| Member      | Creates, modifies and shares workspace content.                                                 |
| Contributor | Creates and modifies content but does not manage permissions.                                   |
| Viewer      | Views content but does not modify it; underlying data access depends on additional permissions. |

## Item and compute permissions

Item permissions grant targeted access to one lakehouse or warehouse without providing access to the whole workspace. Depending on the item, additional permissions may allow SQL endpoint access, Spark/OneLake access or semantic-model report building.

### Least privilege

Prefer item-level sharing when a person needs one asset rather than broad collaboration rights across an entire workspace.

## Granular data security

| Control               | Purpose                                                                   |
|-----------------------|---------------------------------------------------------------------------|
| Object-level security | Restricts access to specific tables, views or procedures.                 |
| Row-level security    | Filters rows based on user identity or context.                           |
| Column-level security | Blocks access to selected sensitive columns.                              |
| Dynamic data masking  | Obscures values in query results without changing stored data.            |
| OneLake security      | Applies role-based access to tables and folders across supported engines. |

## Dynamic data masking

### T-SQL

```
ALTER TABLE Customers
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');

ALTER TABLE Customers
ALTER COLUMN PhoneNumber ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-",4)');
```

Masking is not encryption. It should be combined with least privilege and object-level controls because highly privileged query users may still infer data.

### Row-level security

RLS normally uses an inline table-valued predicate function and a security policy bound to the target table. The engine automatically applies the filter to user queries.

### T-SQL

```
CREATE SCHEMA sec;
GO
CREATE FUNCTION sec.tvf_TenantPredicate(@TenantName nvarchar(100))
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS result WHERE @TenantName = USER_NAME();
GO
CREATE SECURITY POLICY sec.SalesPolicy
ADD FILTER PREDICATE sec.tvf_TenantPredicate(TenantName) ON dbo.Sales
WITH (STATE = ON);
```

## Column-level permissions

### T-SQL

```
CREATE ROLE Receptionist AUTHORIZATION dbo;
GRANT SELECT ON dbo.Patients TO Receptionist;
DENY SELECT ON dbo.Patients (MedicalHistory) TO Receptionist;
```

## OneLake security roles

A custom OneLake security role defines data scope, permission level, members and optional row or column constraints. The DefaultReader role grants broad read access, so users must be removed from it when a restricted custom role is intended to limit access.

### Permission rule

DENY overrides GRANT. Applications that only call stored procedures should normally receive EXECUTE permission rather than direct table access.

## CHAPTER 8

# Monitoring and performance

*Track reliability, capacity consumption and query behaviour.*

## Monitor Hub

The Monitor Hub provides a central view of activity across Fabric. It brings together pipeline runs, Dataflow executions, semantic-model refreshes, Spark jobs, notebooks and other supported operations.

| Monitoring task              | What to inspect                                                       |
|------------------------------|-----------------------------------------------------------------------|
| Pipeline or Dataflow failure | Status, start/end time, duration, activity details and retry history. |
| Spark issue                  | Application progress, resource use, task execution and logs.          |
| Semantic-model refresh       | Refresh result and retry attempts.                                    |
| Eventstream issue            | Ingestion throughput, status and processing behaviour.                |
| Historical troubleshooting   | Compare current performance with previous successful runs.            |

## Monitoring best practices

18. Define the processes and metrics that matter.
19. Collect history to establish normal behaviour.
20. Review logs and metrics regularly.
21. Investigate deviations quickly.
22. Use findings to remove bottlenecks and improve reliability.

## Warehouse capacity monitoring

Fabric capacity is measured using Capacity Units. Warehouse reads, writes, queries and OneLake file operations consume capacity. The Fabric Capacity Metrics app helps administrators analyse utilisation and identify throttling.

## Current warehouse activity

| DMV                     | Purpose                       |
|-------------------------|-------------------------------|
| sys.dm_exec_connections | Active warehouse connections. |
| sys.dm_exec_sessions    | Authenticated user sessions.  |
| sys.dm_exec_requests    | Currently executing requests. |

Joining these DMVs helps identify who is running a query, what command is executing and how long it has been active.

## Historical Query Insights

| View                                 | Purpose                               |
|--------------------------------------|---------------------------------------|
| queryinsights.exec_requests_history  | Completed SQL request history.        |
| queryinsights.long_running_queries   | Queries ranked by execution duration. |
| queryinsights.frequently_run_queries | Queries ranked by execution count.    |
| queryinsights.exec_sessions_history  | Historical session information.       |

### Operational note

Historical query information may not appear immediately. Workspace Admin permissions are required to view other users' queries or terminate sessions with KILL.

## CHAPTER 9

# CI/CD and lifecycle management

*Version, test and promote Fabric solutions safely.*

## CI/CD concepts

| Concept                | Meaning                                                                                                      |
|------------------------|--------------------------------------------------------------------------------------------------------------|
| Continuous integration | Developers frequently commit and merge changes so issues are identified early.                               |
| Continuous delivery    | Validated changes are automatically prepared or deployed to a staging environment before production release. |
| Continuous deployment  | Changes that pass testing are automatically released to production.                                          |

## Three lifecycle pillars in Fabric

| Pillar               | Purpose                                                          |
|----------------------|------------------------------------------------------------------|
| Git integration      | Version history, branching, collaboration and rollback.          |
| Deployment pipelines | Promote content through Development, Test and Production stages. |
| Fabric REST APIs     | Automate Git synchronisation, status checks and deployments.     |

## Git integration workflow

23. Connect a development workspace to a GitHub or Azure DevOps repository and branch.
24. Develop in an isolated branch or workspace instead of overwriting shared work.
25. Commit workspace changes to the feature branch.
26. Open a pull request and merge approved changes into the main branch.
27. Synchronise the shared development workspace with the updated branch.

### Common confusion

Git integration synchronises a workspace with source control. A deployment pipeline promotes Fabric items between environment workspaces. These are complementary but different processes.

## Deployment pipelines

Deployment pipelines assign workspaces to lifecycle stages such as Development, Test and Production. Deploying from one stage to another clones or updates supported items in the target environment.

- A common clean pattern is to connect only the Development workspace to Git.
- Use Git for version control and pull-request review.
- Use deployment pipelines to promote validated content to Test and Production.

## Automation with Fabric APIs

| API area                 | Examples                                                       |
|--------------------------|----------------------------------------------------------------|
| Git integration APIs     | Commit changes, pull updates and check synchronisation status. |
| Deployment pipeline APIs | List stage items and deploy stage content programmatically.    |

### Lifecycle principle

Separate development, testing and production. Use repeatable automation to reduce manual error and keep releases auditable.

## CHAPTER 10

## Final revision tables and scenario guide

*Use these summaries for quick recall before the exam.*

### Which Fabric component should you choose?

| Scenario                                                       | Best starting choice       |
|----------------------------------------------------------------|----------------------------|
| Move large volumes of data with minimal transformation         | Copy Data activity         |
| Apply low-code Power Query transformations                     | Dataflow Gen2              |
| Coordinate multiple activities with dependencies and schedules | Data pipeline              |
| Perform complex large-scale transformations                    | Spark notebook             |
| Store raw and curated Delta tables                             | Lakehouse                  |
| Build a relational star schema with full T-SQL DML             | Fabric Data Warehouse      |
| Ingest and route streaming events                              | Eventstream                |
| Store and query high-velocity time-series data                 | Eventhouse / KQL database  |
| Show auto-refreshing operational visuals                       | Real-Time Dashboard        |
| Trigger actions when a live condition is met                   | Activator                  |
| Read OneLake data directly through Power BI                    | Direct Lake semantic model |

### High-value comparison sheet

| Concept A              | Concept B         | Key distinction                                                 |
|------------------------|-------------------|-----------------------------------------------------------------|
| Dataflow Gen2          | Data pipeline     | Transformation worker vs orchestration manager.                 |
| Copy Data              | Dataflow Gen2     | High-speed movement vs visual transformation.                   |
| Lakehouse              | Warehouse         | Spark + files + Delta tables vs relational SQL-led warehouse.   |
| SQL analytics endpoint | Warehouse         | Read-only lakehouse SQL vs read-write transactional T-SQL.      |
| Eventstream            | Eventhouse        | Ingest/transform/route in motion vs store/query real-time data. |
| Pipeline parameter     | Pipeline variable | Runtime input vs value stored or changed during a run.          |
| Optimize Write         | OPTIMIZE          | Prevent small files during writes vs compact existing files.    |
| V-Order                | Partitioning      | File-level read optimisation vs physical folder layout.         |
| Workspace role         | Item permission   | Broad workspace collaboration vs targeted item access.          |

| Concept A       | Concept B           | Key distinction                                          |
|-----------------|---------------------|----------------------------------------------------------|
| Git integration | Deployment pipeline | Source control synchronisation vs environment promotion. |

## Delta and Lakehouse checklist

- Delta table = Parquet data files + `_delta_log` transaction history.
- ACID, schema enforcement, CRUD and time travel are core Delta benefits.
- Use `OPTIMIZE` for existing small files and `VACUUM` for obsolete files.
- Use partitioning selectively on very large tables with sensible filter columns.
- Bronze is raw, Silver is validated, Gold is business-ready.
- The lakehouse SQL analytics endpoint is read-only for table data.

## Warehouse checklist

- Facts hold measurements; dimensions provide context.
- Business keys come from sources; surrogate keys are generated for warehouse relationships.
- Load dimensions before facts when facts require surrogate-key lookups.
- Type 1 SCD overwrites; Type 2 inserts a new historical row.
- Use full loads for simplicity and incremental loads for efficiency.
- Views standardise reusable query logic; measures standardise business calculations.

## Real-time checklist

- Event = one occurrence; stream = continuous sequence of events.
- Eventstream handles in-flight ingestion, transformation and routing.
- Eventhouse stores high-velocity time-series data in KQL databases.
- Filter early and project only required columns in KQL.
- Materialized views precompute recurring aggregations.
- Activator monitors objects and properties, applies rules and triggers actions.

## Security checklist

- Apply least privilege at workspace, item, compute and data layers.
- RLS filters rows; CLS blocks columns; masking obscures displayed values.
- `DENY` overrides `GRANT`.
- OneLake security roles can restrict tables, folders, rows and columns across supported engines.
- Remove users from `DefaultReader` when a restricted OneLake role must actually limit access.

## Closing note

The DP-700 exam rewards component selection as much as feature knowledge. For each scenario, identify the data shape, transformation complexity, latency requirement, preferred language, security boundary and operational lifecycle. Then choose the Fabric item that best matches those constraints.

### Final revision method

For every practice question, ask: What is the source? Where should data land? Is transformation low-code or Spark-scale? Is the workload batch or streaming? Who needs access? How will it be monitored and deployed?





## About this guide

This publication is a condensed and professionally edited version of personal DP-700 preparation notes. It is designed to help data professionals revise Microsoft Fabric data engineering concepts and compare commonly confused components.

Primary learning basis: Microsoft Learn DP-700 training modules, as summarised in the supplied preparation document.

Author: Jitendra Singh Malik

Edition: August 2026

### Sharing note

You may share this guide as an independent learning resource. Keep the disclaimer and source acknowledgement visible so readers understand that it is not official Microsoft documentation.

# Seven-day revision plan

*A practical final-week sequence for revising the guide without rereading every page.*

| Day   | Focus                                                | Revision goal                                                                 | Done                     |
|-------|------------------------------------------------------|-------------------------------------------------------------------------------|--------------------------|
| Day 1 | Fabric foundations, OneLake and workload selection   | Explain OneLake, workspaces and the main Fabric workloads without notes.      | <input type="checkbox"/> |
| Day 2 | Data Factory, Copy activity, Dataflows and pipelines | Practise choosing the correct ingestion or orchestration component.           | <input type="checkbox"/> |
| Day 3 | Spark, DataFrames, Spark SQL and notebooks           | Translate familiar SQL operations into DataFrame and Spark SQL patterns.      | <input type="checkbox"/> |
| Day 4 | Lakehouse, Delta Lake and medallion architecture     | Review Delta structure, optimisation, time travel and Bronze/Silver/Gold.     | <input type="checkbox"/> |
| Day 5 | Warehouse modelling and loading                      | Revise facts, dimensions, surrogate keys, SCDs, CTAS and incremental loads.   | <input type="checkbox"/> |
| Day 6 | Real-Time Intelligence and security                  | Compare Eventstream, Eventhouse, dashboards, Activator and permission layers. | <input type="checkbox"/> |
| Day 7 | Monitoring, CI/CD and final scenario tables          | Complete practice questions and review every common-confusion box.            | <input type="checkbox"/> |

**Recommended technique:** After each day, write three scenario questions of your own and explain why the selected Fabric component is better than the alternatives.

# Personal revision notes

*Use this page to record concepts that require another review, mistakes from practice tests, and your own workplace examples.*

## Components I still confuse

---

---

---

---

---

## Important commands or syntax

---

---

---

---

---

## Practice-question mistakes

---

---

---

---

---

## Workplace examples that help me remember

---

---

---

---

---